

AMV3D: Adaptive Multi-Layer Volumetric 3D for DICOM

Leonardo Locatelli
Adapta Studio
Milano, Italy
leonardo.locatelli@adapta.studio

Matteo Valoriani
FifthIngenium
Milano, Italy
Dipartimento di Elettronica,
Informazione e Bioingegneria
Politecnico di Milano
Milano, Italy
matteo.valoriani@fifthingenium.com

Manuel Pezzera
FifthIngenium
Milano, Italy
manuel.pezzera@fifthingenium.com

Simona Perotto
Dipartimento di Matematica
Politecnico di Milano
Milano, Italy
ADAPTA studio
Milano, Italy
simona.perotto@polimi.it

Francesco Clasadonte
Fifthingenium
Milano, Italy
francesco.clasadonte@fifthingenium.com

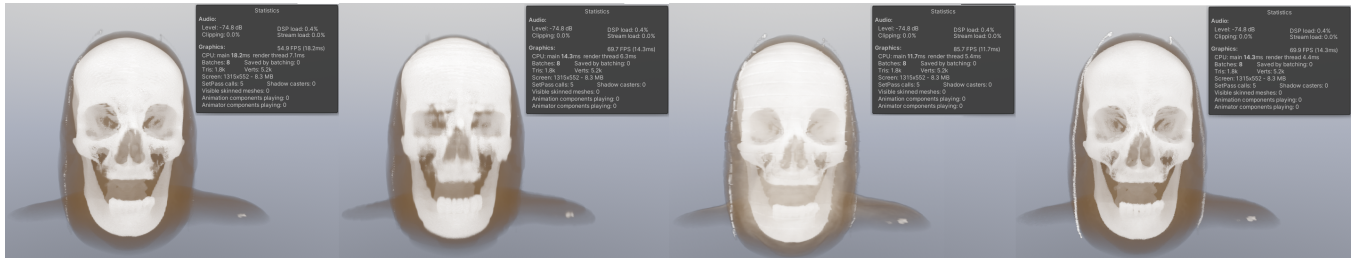


Figure 1: AMV3D examples: (a) Unity 3D texture full; (b) 25% Unity 3D texture; (c) AMV3D at $_MaxDepth = 8$; (d) NanoVDB file

Abstract

Real-time volumetric visualization in Unity is hampered by slow loads, high memory use, and wasted sampling in homogeneous regions; common formats (Unity 3D textures, NRRD, OpenVDB, DICOM-to-texture) need heavy preprocessing or lack GPU-ready multi-resolution access. AMV3D couples a volumetric format with an XRrenderer: an adaptive Morton-linearized octree stores per-node averages and negative-index fallbacks for fast ancestor lookup. An adaptive DVR shader scales ray steps to node size, skipping empty areas.

CCS Concepts

• **Human-centered computing** → **Scientific visualization**; • **Computing methodologies** → **Volumetric models**; **Ray tracing**; **Graphics file formats**; **Virtual reality**; **Image manipulation**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters '25, Hong Kong, Hong Kong

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2134-2/25/12

<https://doi.org/10.1145/3757374.3771777>

ACM Reference Format:

Leonardo Locatelli, Matteo Valoriani, Manuel Pezzera, Simona Perotto, and Francesco Clasadonte. 2025. AMV3D: Adaptive Multi-Layer Volumetric 3D for DICOM. In *SIGGRAPH Asia 2025 Posters (SA Posters '25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3757374.3771777>

1 Introduction

Volumetric medical imaging, such as CT scans stored in DICOM (Digital Imaging and Communications in Medicine) format, is crucial for modern diagnostics. Yet, rendering these large 3D datasets interactively and with high fidelity remains challenging, often causing slow loading and high GPU memory usage, especially on low-power AR/VR devices. Existing solutions for handling volumetric medical data include Unity 3D textures [Documentation 2025], NRRD files [Kindlmann 2010], OpenVDB/NanoVDB grids [Museth 2013; Museth et al. 2021], and direct DICOM pipelines [Lavik 2025]. While each one has strengths in specific scenarios, most require heavy preprocessing or lack integrated, GPU-ready multi-resolution access.

2 AMV3D: Format & Volumetric Rendering

AMV3D is a volumetric data format paired with a real-time rendering pipeline for medical XR. It is automatically built from orthogonal

DICOM stacks, preserving spatial accuracy and intensity ranges (e.g., Hounsfield units). The file doubles as persistent storage and a GPU-ready layout, enabling direct ingestion into interactive renderers without preprocessing. Two interchangeable encodings are provided: **ASCII** [Leonardo Locatelli 2025] and **binary**.

Data are organized as an adaptive, *unbalanced* octree [Sundar et al. 2008] with *all* levels stored explicitly in-file, enabling on-demand access to any resolution without recomputation. During construction, the volume is recursively subdivided until a homogeneity criterion is met or `maxDepth`/`maxElems` limits are reached. Empty or uniform regions store *negative fallback indices* (sentinel floats) that reference the nearest ancestor with valid data, allowing constant-time empty-node resolution and efficient skipping at render time.

Nodes are serialized per depth in Morton (Z-order) order [Akenine-Möller et al. 2018], enabling $O(1)$ array indexing while preserving spatial locality. Morton codes are computed via bit interleaving using precomputed masks (*magic numbers*) [Akenine-Möller et al. 2018]. Given a normalized position $\mathbf{p}$ and target depth, the code plus a cumulative-offset table yields the node address; if empty, the stored negative fallback index redirects to the closest valid ancestor.

2.1 Volumetric rendering in Unity

Unity lacks a built-in framework for volumetric medical visualization; we therefore developed custom shaders, extending the open-source foundation in [Lavik 2025]. AMV3D implements *adaptive DVR* (inspired by [Schneider 2017]) that modulates ray-march step size by the local octree structure: coarse nodes are traversed with larger strides, while fine nodes use smaller steps for higher fidelity near boundaries.

Moreover, by storing octree levels densely in Morton order, the renderer adjusts `_MaxDepth` at runtime, extending Unity’s 3D mipmaps into a volume-aware hierarchy for efficient, device-adaptive AR/VR rendering.

3 Performance Evaluation

We evaluate a representative case—the Visible Human *female head* (Harvard collection) [Harvard Dataverse 2023]—benchmarking AMV3D against Unity’s direct DICOM pipeline and OpenVDB and NanoVDB. The dataset comprises 234 axial 512×512 DICOM slices (≈ 61.34 million voxels) and constitutes our most demanding case in terms of performance and memory. Using AMV3D at `_MaxDepth = 8`, the effective voxel count reduces to 3 million non-empty voxels. The tests are conducted on: a standard laptop (Intel Core i7 11th-gen CPU, 32 GB RAM, *integrated Intel graphics*) and a Meta Quest 3 headset running standalone XR. We use Unity 2022.3.39 (LTS) for stability: URP (forward), Linear color, OpenXR, single-pass instanced. Graphics backends: Direct3D (PC) and Vulkan (Quest). Dynamic resolution and fixed-foveated rendering are *disabled* for comparability. Performance is measured on PC with the Unity Profiler (GPU Recorder) over 30 s after a 5 s warm-up, and on Quest with the on-device OVR Metrics Tool. We report median frame time (ms) and FPS. Opening time is measured from volume load to the first rendered frame. We use identical transfer functions and camera paths for all baselines (Unity Texture3D full/25%, AMV3D, OpenVDB/NanoVDB). We compute 2D, RMSE and slice-wise SSIM (Structural Similarity Index Measure, Gaussian 11×11 window ($\sigma=1.5$),

Table 1: Comparison results on the *female head* test case.

	AMV3D	Texture3D Full / 25%	OpenVDB NanoVDB
RMSE	0.084	0 / 0.064	0.085
SSIM	0.91	0.93	0.92
File Size (MB)	74	120	50 / 69
Δ RAM (GB) Allocated	+0.40	+1.72 / +1.12	+0.23
Δ RAM (GB) Reserved	+1.80	+2.72 / +1.90	+1.50
PC: Opening (s)	0.1	25	0.1
PC: Frame time (ms)	11.76	18.18 / 15.38	12.82
PC: FPS	85	55 / 65	78
Quest: Opening (s)	0.1	OOM / 35	0.1
Quest: Frame time (ms)	27.78	- / 33.33	76.92
Quest: FPS	36	- / 30	13

$C_1=(0.01L)^2$, $C_2=(0.03L)^2$, $L=1$) on matched frontal views against the full-resolution reference after per-volume linear normalization to $[0, 1]$. Table 1 reports SSIM in physical space at each method’s native LOD, masking air/metal ($HU \in (0, 3500)$). Figure 1 shows visual outputs in order: (a) the benchmark yielded by Unity Texture3D; (b) the Texture3D downsampled to 25%; (c) the AMV3D output; (d) the NanoVDB file. The four images reveal only minor differences across the different approaches.

Visual appearance. RMSE and SSIM values against the full resolution benchmark indicate minimal overall differences: the 25% downsampled Texture3D shows slightly lower RMSE due to smoothing, but loses sharpness in fine and high-contrast structures that AMV3D preserves at appropriate LOD.

Disk & Memory footprint. In terms of file size, AMV3D requires 74 MB, corresponding to **38% reduction** compared to the full-resolution Texture3D (120 MB) and **7% increase** relative to NanoVDB (69 MB). Memory benchmarks show that AMV3D increases allocated RAM by **+0.40 GB**, which is **-77%/-64%** relative to Texture3D full/25% downsampled, and +0.17 GB vs NanoVDB (+0.23 GB). Reserved RAM shows a similar trend.

Runtime Performance. On PC, AMV3D loads almost instantaneously and achieves **11.76 ms (85 FPS)**, outperforming both full-resolution Unity Texture3D and 25% downsampled textures (**18.18/15.38 ms**, 55/65 FPS). AMV3D is also slightly faster than NanoVDB (**12.82 ms**, 78 FPS). On Quest, AMV3D reaches **27.78 ms (36 FPS)**, outperforming Texture3D 25% downsampled (**33.33 ms**, 30 FPS) and NanoVDB (**76.92 ms**, 13 FPS) while Texture3D full fails with **out-of-memory (OOM)**. Unlike NanoVDB, AMV3D retains the full multi-resolution hierarchy in memory, trading a modest RAM overhead for instant LOD switching and deterministic quality control on XR devices.

4 Conclusions

We presented AMV3D, an adaptive pipeline for medical volume rendering that combines multi-resolution hierarchy, empty-space skipping, and adaptive sampling to preserve fidelity with high performance. Future work will integrate its structure with OpenVDB and NanoVDB.

References

- Harvard Dataverse. 2023. *Visible Human Project CT Datasets Male and Female*. doi:10.7910/DVN/3JDZCT
- Tomas Akenine-Möller, Eric Haines, Naty Hoffman, Angelo Pesce, Michaël Iwanicki, and Sebastien Hillaire. 2018. *Real-Time Rendering, Fourth Edition*. A K Peters/CRC Press.
- Unity Documentation. 2025. Density Volume Support in Unity URP. Unity Manual. <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/api/index.html>
- Gordon Kindlmann. 2010. The NRRD File Format: Nearly Raw Raster Data. Teem Documentation. <http://teem.sourceforge.net/nrrd/format.html> Defines the NRRD volumetric format, with flexible header and data layout.
- Magnus Lavik. 2025. UnityVolumeRendering. GitHub repository. <https://github.com/mlavik1/UnityVolumeRendering> Open-source Unity project for volume rendering, supporting NRRD, DICOM, and other volumetric formats.
- Simona Perotto Manuel Pezzera Leonardo Locatelli, Matteo Valoriani. 2025. AMV3D: Adaptive Medical Volume 3D Rendering Pipeline. GitHub repository. <https://github.com/LeoLoca93/AMV3D> Adaptive volumetric rendering framework for medical data featuring multi-resolution hierarchy, empty-space skipping, and adaptive DVR visualization..
- Ken Museth. 2013. VDB: High-Resolution Sparse Volumes with Dynamic Topology. *ACM Transactions on Graphics (TOG)* 32, 3 (2013), 27:1–27:22. https://www.museth.org/Ken/Publications_files/Museth_TOG13.pdf Introduces the hierarchical sparse data structure behind OpenVDB..
- Ken Museth, Jeff Budberg, Matheus Gadelha, et al. 2021. NanoVDB: A Compact and GPU-Friendly Representation for OpenVDB. NVIDIA Research. <https://research.nvidia.com/labs/prl/nanovdb/nanovdb2021.pdf>
- Andrew Schneider. 2017. The Real-Time Volumetric Cloudscapes of Horizon Zero Dawn. In *ACM SIGGRAPH 2017 Courses: Advances in Real-Time Rendering in Games*. ACM. <http://advances.realtimerendering.com/s2017/> Course notes.
- Hari Sundar, Rahul S. Sampath, and George Biros. 2008. Bottom-Up Construction and 2:1 Balance Refinement of Linear Octrees in Parallel. *SIAM Journal on Scientific Computing* 30, 5 (2008), 2675–2708. Describes linear octrees using Morton encoding for scalable parallel construction..